

Mastermind Game Refactoring and Enhancement

<https://gemini.google.com/app/fdeb1dce1023a256>

User prompt: Before changing the code Construct codes and guesses containing repeated symbols. Reason about how many times a single code occurrence should be allowed to contribute to feedback. Task 1 — Correct duplicate-aware feedback Implement feedback so each code position can be counted at most once. Exact matches must be resolved before partial matches. Done when: repeated-symbol cases produce the correct exact and partial counts. Task 2 — Complete game lifecycle Add clear win/loss handling, preserve guess history, enforce the guess limit, and prevent state changes after the game ends. Task 3 — Difficulty modes Add difficulty choices that vary code length, symbol range, and allowed guesses. The feedback algorithm must work for all supported configurations. Task 4 — History and input robustness Present a readable guess history and reject malformed guesses without consuming turns. Feedback must correspond to one accepted guess only.

Response: To correctly handle repeated symbols, a single code symbol can only contribute to feedback once, and exact positional matches must be resolved first to prevent partial matches from consuming a symbol meant for an exact match.

For example, if the code is 1234 and the guess is 1111:

- The first 1 (index 0) is an exact match.
- The remaining guess symbols [1, 1, 1] must be checked against the remaining unmatched code symbols [2, 3, 4].
- Because there are no 1s left in the remaining code, the partial count is 0. The original `logic.py` incorrectly evaluated `ch in code` for every single guess symbol, yielding erroneous partial matches.

If the code is 2234 and the guess is 1222:

- The 2 at index 2 is an exact match.
- The remaining guess symbols [1, 2, 2] are evaluated against the remaining code symbols [2, 3].
- Only one 2 exists in the remaining code, so it is consumed. This yields 1 exact match and 1 partial match.

Here are the updated files to implement accurate feedback, dynamic difficulty, readable history, and full lifecycle handling.

logic.py

```
def feedback(code, guess):
    exact = 0
    unmatched_code = []
    unmatched_guess = []

    # First pass: Identify and count exact matches
    for c, g in zip(code, guess):
        if c == g:
            exact += 1
        else:
            unmatched_code.append(c)
            unmatched_guess.append(g)

    partial = 0
    # Second pass: Identify partial matches using only unmatched symbols
    for g in unmatched_guess:
        if g in unmatched_code:
            partial += 1
            unmatched_code.remove(g) # Consume the code symbol

    return exact, partial
```

game.py

```
import random
from logic import feedback

class Mastermind:
    def __init__(self, mode="normal"):
        modes = {
            "easy": {"length": 4, "symbols": "123456", "turns": 10},
            "normal": {"length": 5, "symbols": "12345678", "turns": 12},
            "hard": {"length": 6, "symbols": "123456789", "turns": 15}
        }
        settings = modes.get(mode, modes["normal"])

        self.length = settings["length"]
        self.symbols = settings["symbols"]
        self.turns = settings["turns"]

        # Generates a code based on selected difficulty rather than hardcoding 4 digits[cite: 1]
        self.code = [str(random.choice(self.symbols)) for _ in range(self.length)]
        self.history = []
        self.active = True
```

```

def run(self):
    print(f"\nMastermind – enter {self.length} digits from {self.symbols[0]} to {self.symbols[-1]}.")

    while self.active and self.turns > 0:
        if self.history:
            print("\n--- Guess History ---")
            for i, (g, e, p) in enumerate(self.history, 1):
                print(f"Turn {i}: {g} | Exact: {e}, Partial: {p}")
            print("-----")

        raw = input(f"{self.turns} turns left > ").strip()

        if raw.lower() == "q":
            print("Game aborted.")
            self.active = False
            return

        # Validates length and allowed characters before consuming a turn[cite: 1]
        if len(raw) != self.length or any(ch not in self.symbols for ch in raw):
            print(f"Invalid! Enter exactly {self.length} digits from {self.symbols[0]} to {self.symbols[-1]}.")
            continue

        guess = list(raw)
        exact, partial = feedback(self.code, guess)

        self.history.append((raw, exact, partial))
        self.turns -= 1

        print(f"Result -> Exact: {exact}, Partial: {partial}")

        if exact == self.length:
            print("\nCongratulations! You cracked the code!")
            self.active = False
            return

    if self.active:
        print(f"\nGame over! Out of turns. The code was {''.join(self.code)}")
        self.active = False

```

main.py

```

from game import Mastermind

if __name__ == "__main__":
    print("Welcome to Mastermind!")
    print("Select Difficulty: (1) Easy (2) Normal (3) Hard")

    choice = input("> ").strip()
    mode_map = {"1": "easy", "2": "normal", "3": "hard"}
    mode = mode_map.get(choice, "normal")

    # Passes the chosen mode instead of relying on an empty default initialization[cite: 3]
    game = Mastermind(mode=mode)
    game.run()

```

Key Enhancements

- **Task 1 (Duplicate-Aware Feedback):** `logic.py` divides the validation into two distinct passes. By extracting exact matches first, we safely check remaining guess symbols against unmatched code symbols and `.remove()` matched characters to prevent double-counting.
- **Task 2 (Game Lifecycle):** The introduction of `self.active` cleanly terminates the gameplay loop. A definitive win/loss state is explicitly checked and displayed before state changes are locked out.
- **Task 3 (Difficulty Modes):** `game.py` swaps the rigid generation block for a dictionary mapped to variable lengths, symbol arrays, and guess limits. The `logic.py` loop safely iterates over any sequence length using `zip(code, guess)`.
- **Task 4 (History & Robustness):** `game.py` iterates and prints formatted elements from `self.history` on every valid turn. Input filtering halts malformed submissions using `continue`, ensuring turns are only subtracted when a valid string is evaluated.

User prompt: Required testing Test exact matches, repeated symbols, no matches, mixed exact/partial cases, every difficulty, invalid guesses, final-turn wins, losses, and quitting. give me 5 test cases to test this code

Response: **Test Case 1: Easy Mode (Invalid Guesses & Quitting)**

- **Objective:** Verify difficulty setup, input validation rejection, and the early exit command.

- **Setup:** Launch `main.py`, select 1 (Easy mode: 4 length, 1-6 symbols). Mock `self.code` as `['1', '2', '3', '4']`.
- **Inputs & Expected Outcomes:**
 1. Enter 123 (too short): Prints "Invalid! Enter exactly 4 digits from 1 to 6." Turn count remains 10.
 2. Enter 1239 (out of range): Prints "Invalid!..." Turn count remains 10.
 3. Enter q: Prints "Game aborted" and immediately exits the run loop.

Test Case 2: Normal Mode (Repeated Symbols & Duplicate-Aware Feedback)

- **Objective:** Ensure exact matches are resolved before partials, preventing a single code occurrence from being consumed twice.
- **Setup:** Select 2 (Normal mode: 5 length, 1-8 symbols). Mock `self.code` as `['2', '2', '3', '4', '5']`.
- **Inputs & Expected Outcomes:**
 1. Enter 12222: Returns **Exact: 1, Partial: 1**. (The '2' at index 2 is an exact match. The remaining guess has three '2's, but the unmatched code only has one '2' left at index 0).
 2. Enter 22888: Returns **Exact: 2, Partial: 0**.

Test Case 3: Hard Mode (No Matches & Loss Condition)

- **Objective:** Validate extended symbol range, zero-match feedback, and proper state lock after running out of turns.
- **Setup:** Select 3 (Hard mode: 6 length, 1-9 symbols). Mock `self.code` as `['1', '1', '2', '2', '3', '3']`.
- **Inputs & Expected Outcomes:**
 1. Enter 456789 for the first 14 turns: Each turn accurately prints **Exact: 0, Partial: 0** and appends to the history block.
 2. Enter 456789 on the 15th turn: Prints **Exact: 0, Partial: 0**, followed immediately by "Game over! Out of turns. The code was 112233". The script exits.

Test Case 4: Any Mode (Mixed Feedback & Final-Turn Win)

- **Objective:** Test correct mixed feedback and ensure winning on the absolute last turn triggers the win state without triggering the loss state.
- **Setup:** Select 1 (Easy mode). Mock `self.code` as `['6', '5', '4', '3']`. Simulate 8 previous incorrect guesses (2 turns remaining).
- **Inputs & Expected Outcomes:**
 1. Enter 5644 (Turn 9): Returns **Exact: 1** (the '4'), **Partial: 2** (the '5' and '6').
 2. Enter 6543 (Turn 10): Returns **Exact: 4**. Prints "Congratulations! You cracked the code!" and exits without printing the "Game over! Out of turns." message.

Test Case 5: Complex Shift (Advanced Exact/Partial Collision)

- **Objective:** Stress-test the two-pass logic against a heavily duplicated, shifted sequence.
 - **Setup:** Select 2 (Normal mode). Mock `self.code` as `['8', '1', '8', '2', '8']`.
 - **Inputs & Expected Outcomes:**
 1. Enter 88182:
 - Index 0 matches perfectly (`8 == 8`).
 - Unmatched code pool becomes `['1', '8', '2', '8']`.
 - Unmatched guess pool becomes `['8', '1', '8', '2']`.
 - The remaining guess symbols all exist exactly once in the unmatched code pool.
 - Returns **Exact: 1, Partial: 4**.
-